LAPORAN PRATIKUM
TEKNOLOGI IOT ll



Oleh:

NAMA: SULANA WIJAYA

NPM: 25782113

KELAS: TRI – 3D

Dosen:

Dr. Ir Septafiansyah Dwi Putra, S.T., MT.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

Modul 3: Jaringan Nirkabel dan Komunikasi HTTP

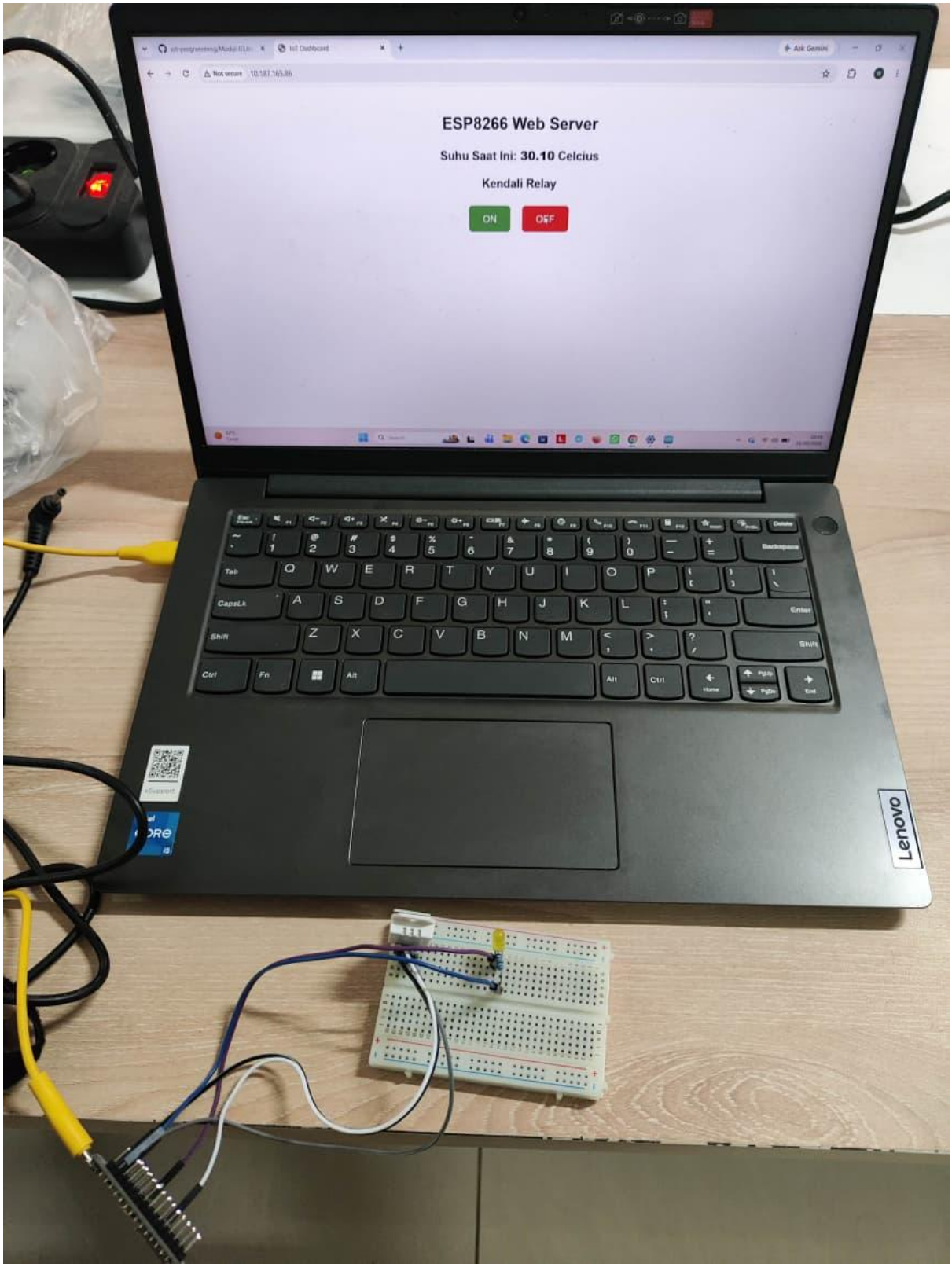
Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)

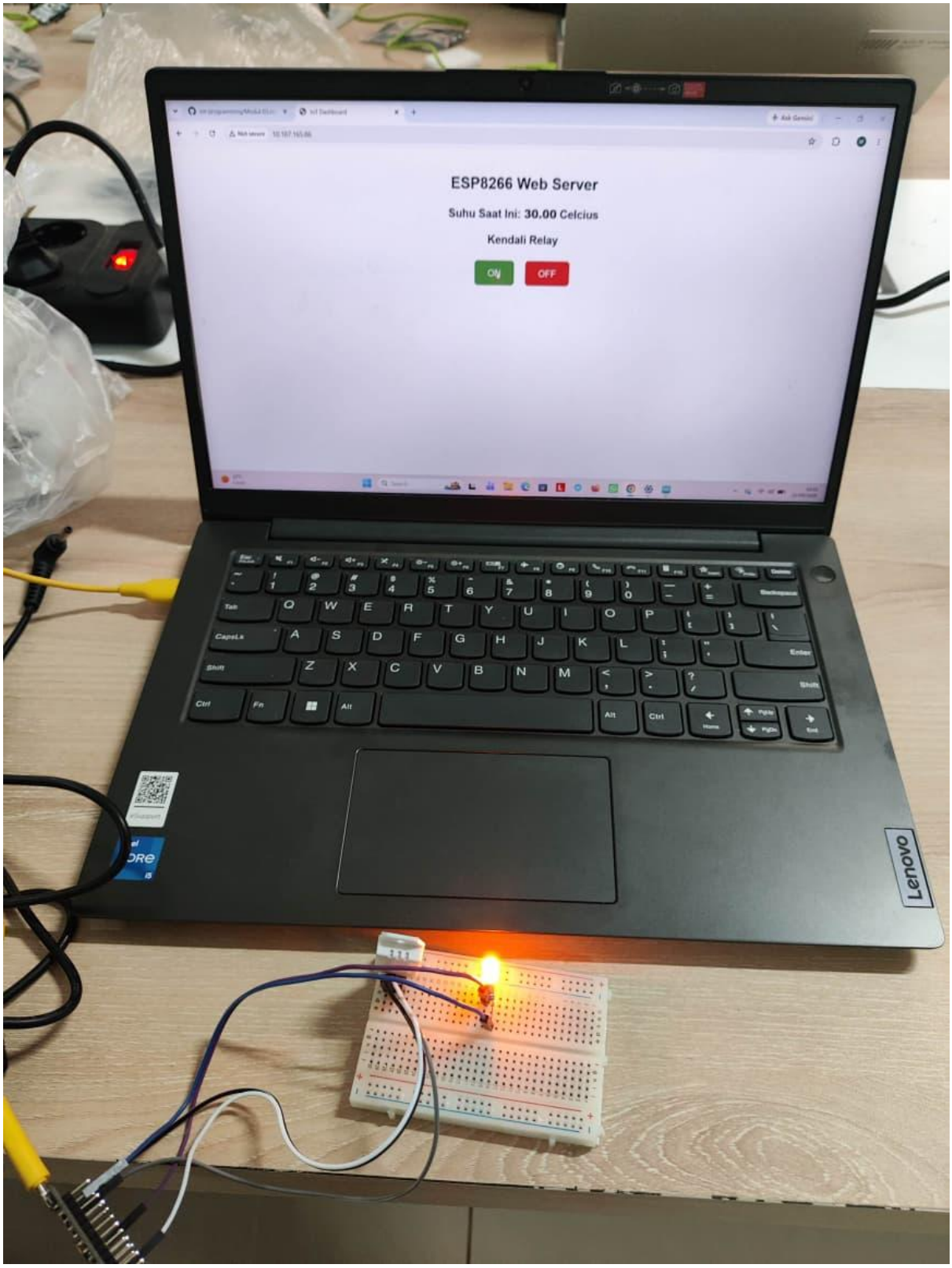
Pastikan komponen berikut tersedia di meja Anda:

- *Board* NodeMCU ESP8266 (1 buah)
- Kabel Micro USB (1 buah)
- *Breadboard* (1 buah)
- Sensor Suhu & Kelembapan (DHT11 atau DHT22) (1 buah)
- Sensor LDR (1 buah)
- Resistor 10k Ohm (1 buah, untuk pembagi tegangan LDR)
- LED 5mm (1 buah)
- Resistor 220 Ohm (1 buah, untuk pembatas arus LED)
- Kabel *Jumper Male-to-Male* dan *Male-to-Female* (secukupnya)

Langkah Kerja Rangkaian:

1. Hubungkan pin DATA / OUT / S DHT ke pin D4 (GPIO 2). (Pastikan VCC DHT ke 3V3, GND ke GND).
2. Hubungkan kaki Anoda (kaki panjang) LED ke pin D6 (GPIO 12). Hubungkan kaki Katoda (kaki pendek) LED ke salah satu kaki Resistor 220 Ohm, lalu hubungkan kaki resistor lainnya ke pin GND NodeMCU. *(Catatan: LED ini dikendalikan oleh pin D6 / GPIO 12 yang pada kode program didefinisikan sebagai relayPin).*

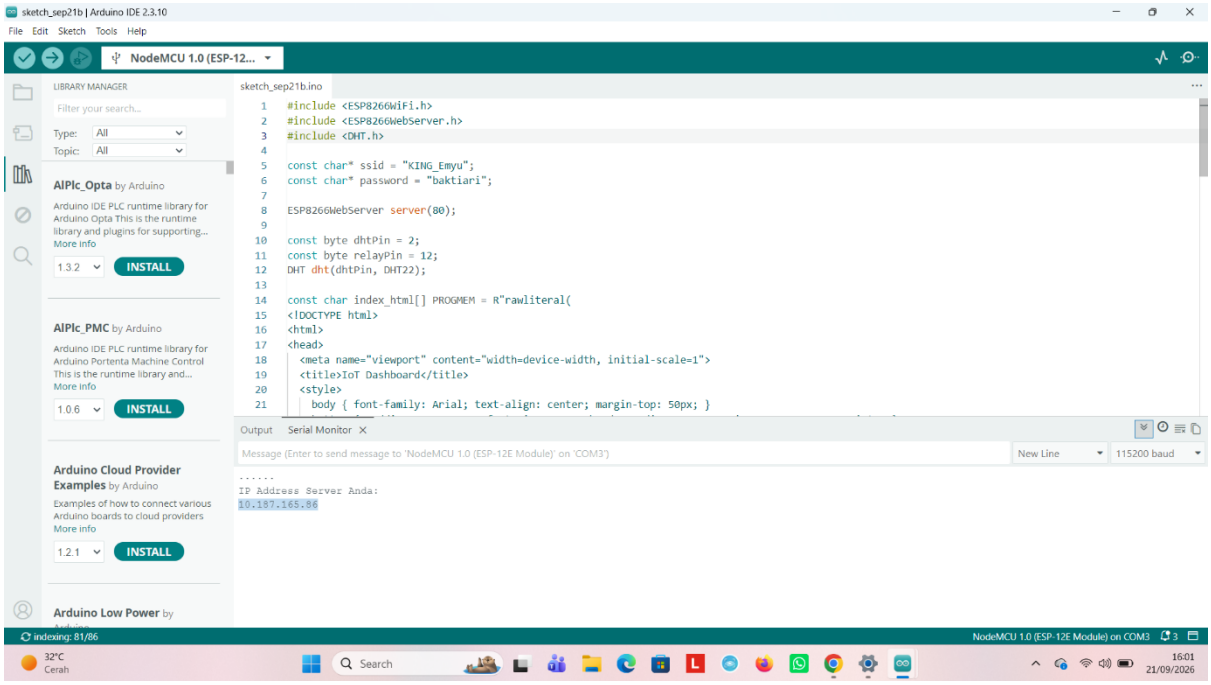

ESP8266 Web Server
Suhu Saat Ini: 30.10 Celcius
Kendali Relay
ON
OFF
Lenovo



Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*). Tuliskan kode program di bawah. Jangan lupa mengganti variabel ssid dan password!

2. *Catatan teknis:* Kita menggunakan teknik Raw String Literal C++ (R"rawliteral(...)rawliteral") untuk menulis kode antarmuka HTML multibaris, sehingga kita tidak repot mengetikkan tanda kutip "" di setiap baris HTML.



```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <DHT.h>

const char* ssid = "NAMA_WIFI_ANDA";
const char* password = "PASSWORD_WIFI_ANDA";

ESP8266WebServer server(80);

const byte dhtPin = 2;
const byte relayPin = 12;
DHT dht(dhtPin, DHT11);

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>IoT Dashboard</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top: 50px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px;
    .btn-on { background-color: #4CAF50; color: white; border: none; }
    .btn-off { background-color: #f44336; color: white; border: none; }
    .sensor-box { font-size: 24px; font-weight: bold; }
  </style>
</head>
<body>
  <h1>ESP8266 Web Server</h1>
```

```
  <div class="sensor-box">
    <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
  </div>
  <h2>Kendali Relay</h2>
  <a href="/relay/on"><button class="btn-on">ON</button></a>
  <a href="/relay/off"><button class="btn-off">OFF</button></a>
</body>
</html>
)rawliteral";

void handleRoot() {
  String html = index_html; // Salin kerangka HTML ke variabel dinamis
  float t = dht.readTemperature();

  // Mengganti teks placeholder %TEMPERATURE% dengan suhu nyata
  if (isnan(t)) {
    html.replace("%TEMPERATURE%", "--");
  } else {
    html.replace("%TEMPERATURE%", String(t));
  }

  server.send(200, "text/html", html);
}

void handleRelayOn() {
  digitalWrite(relayPin, HIGH);
  server.sendHeader("Location", "/");
  server.send(303);
}

void handleRelayOff() {
  digitalWrite(relayPin, LOW);
```

```
  server.sendHeader("Location", "/");
  server.send(303);
}

void setup() {
  Serial.begin(115200);
  pinMode(relayPin, OUTPUT);
  digitalWrite(relayPin, LOW);
  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nIP Address Server Anda: ");
  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.on("/relay/on", handleRelayOn);
  server.on("/relay/off", handleRelayOff);
  server.begin();
}

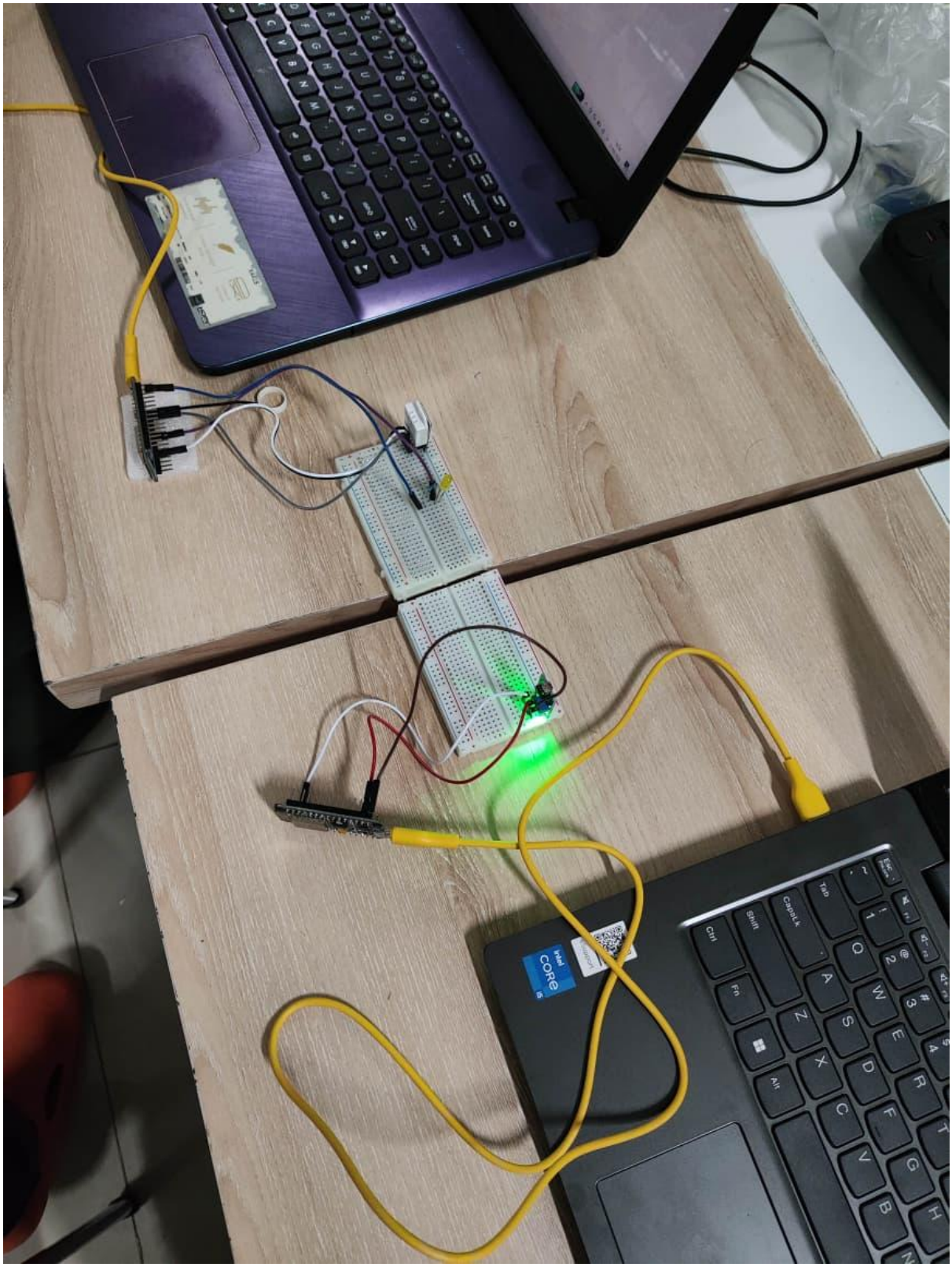
void loop() {
  server.handleClient();
}
```

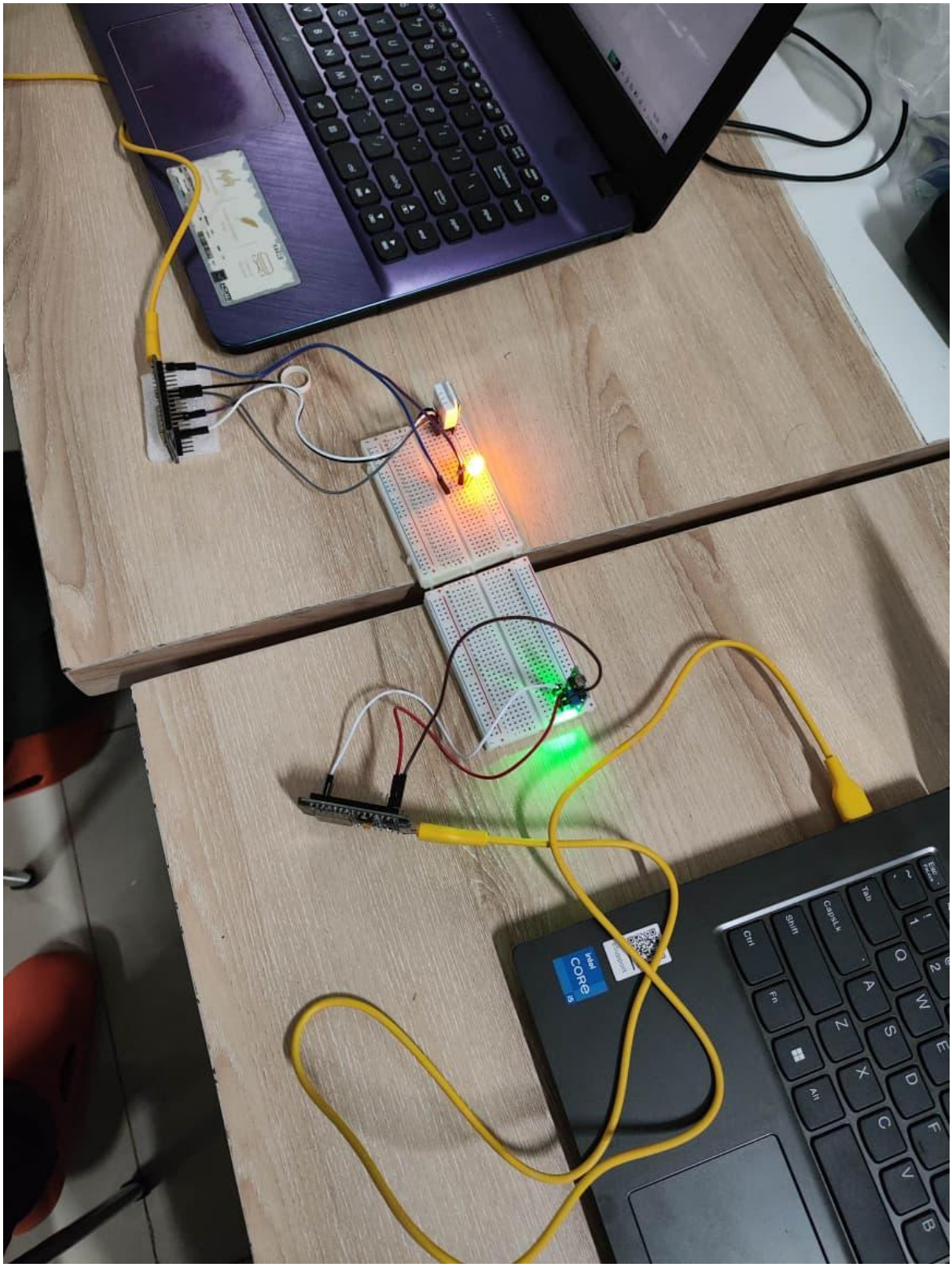
3. Upload kode tersebut. Buka Serial Monitor, dan catat alamat IP yang didapat (contoh: 192.168.43.15).
4. Hubungkan Laptop/HP Anda ke Wi-Fi yang sama, buka *browser* (Chrome/Safari), lalu ketikkan alamat IP tersebut. Klik tombol ON dan OFF di layar untuk melihat LED menyala dan padam.

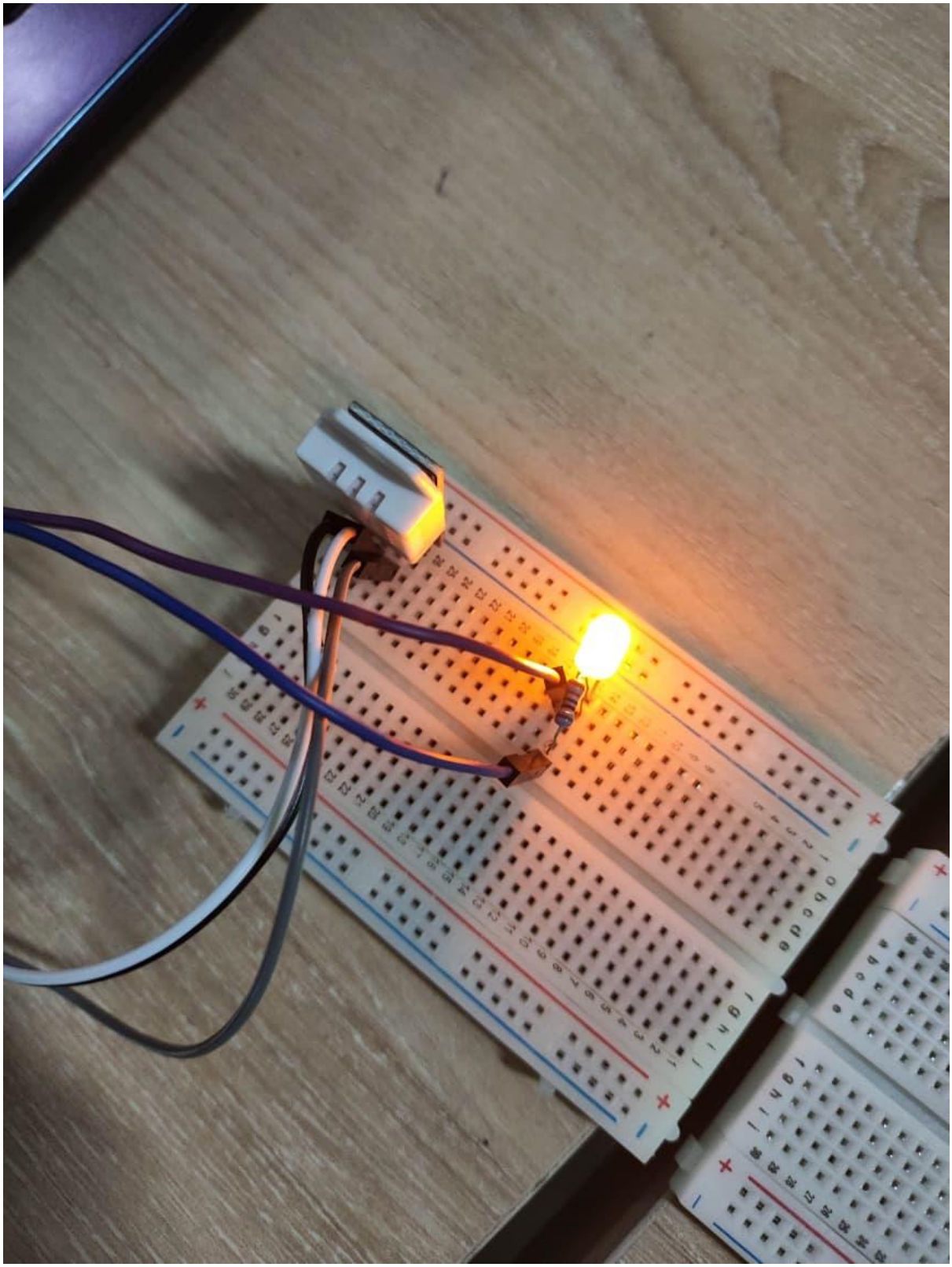
Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)

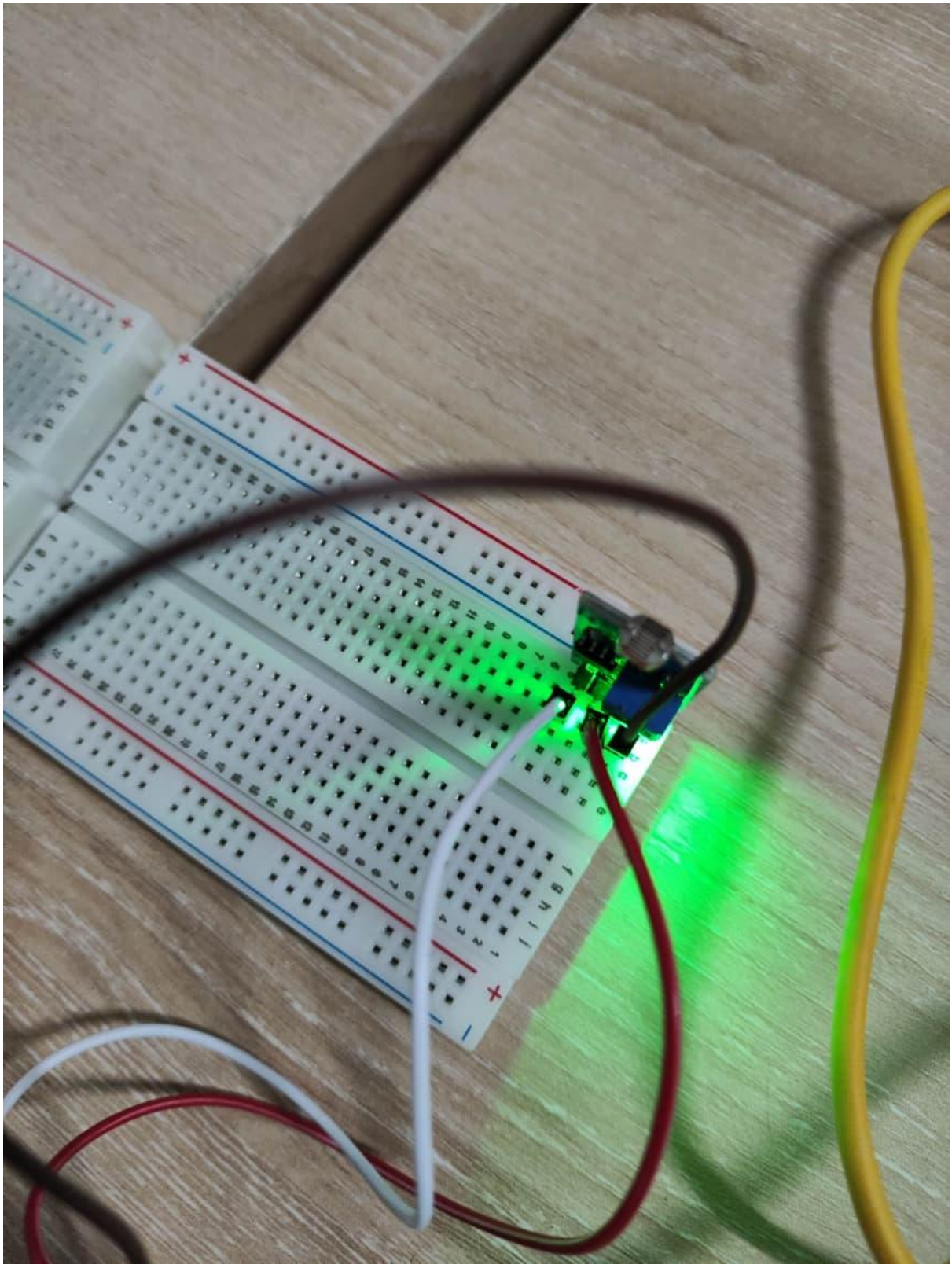
Langkah Kerja Rangkaian:

1. Rangkai sensor LDR dan Resistor 10k Ohm sebagai pembagi tegangan (seperti di Modul 2) lalu hubungkan jalur sinyalnya ke pin A0.


CORE
i5


intel
CORE
i5





Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*).

2. Tuliskan kode program di bawah. Ganti variabel serverName dengan IP Address NodeMCU Praktikum 1 milik teman sebelah Anda. (Pastikan Anda menggunakan Wi-Fi yang sama persis).

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>

const char* ssid = "NAMA_WIFI_ANDA";
const char* password = "PASSWORD_WIFI_ANDA";

const char* serverName = "http://192.168.1.15/relay/on";
const byte ldrPin = A0;

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nClient Terhubung ke Wi-Fi!");
}

void loop() {
  int ldrValue = analogRead(ldrPin);

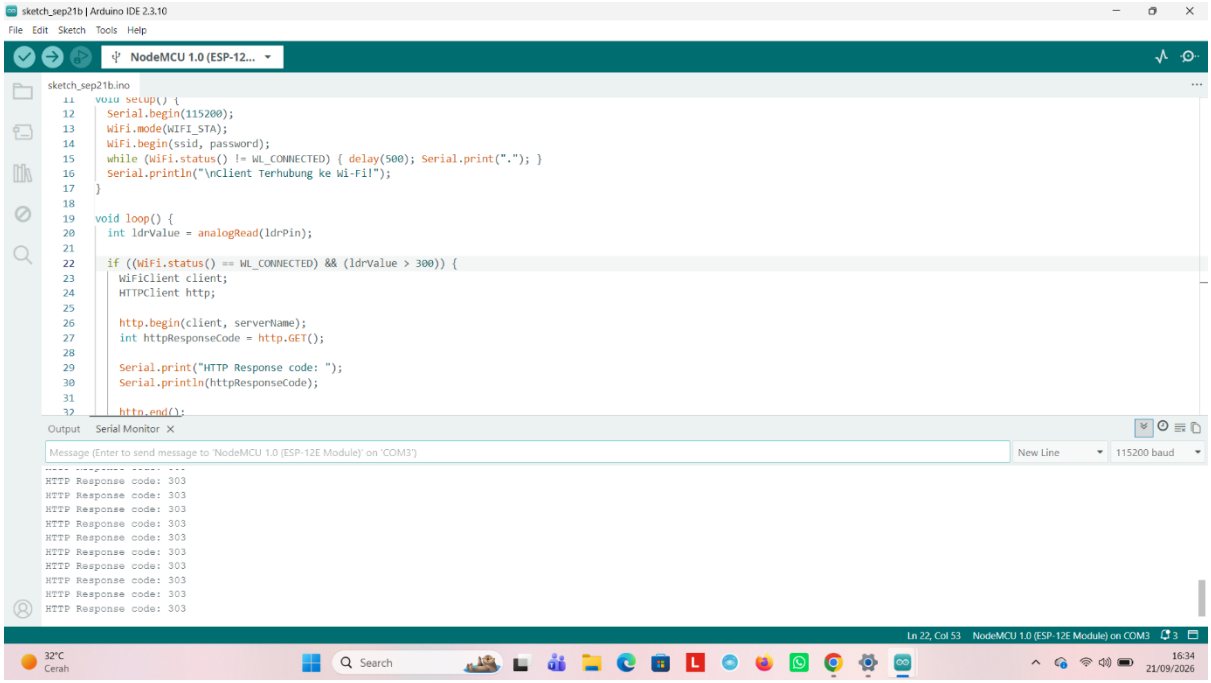
  if ((WiFi.status() == WL_CONNECTED) && (ldrValue < 300)) {
    WiFiClient client;
    HTTPClient http;

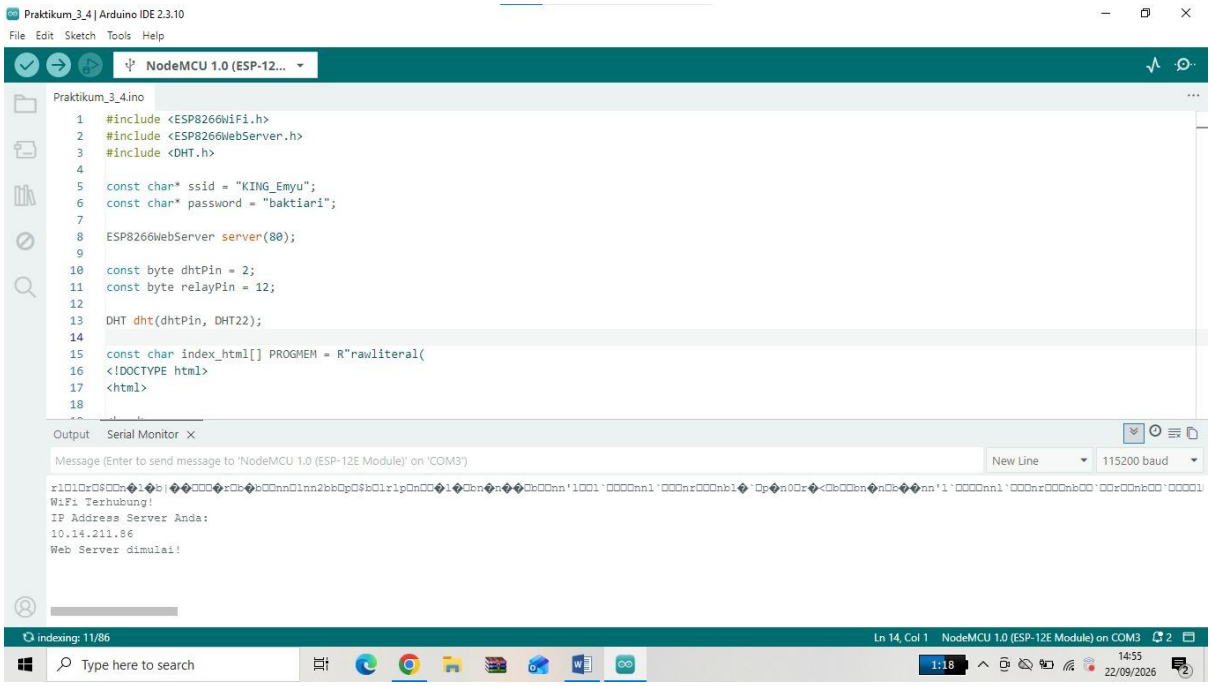
    http.begin(client, serverName);
    int httpResponseCode = http.GET();

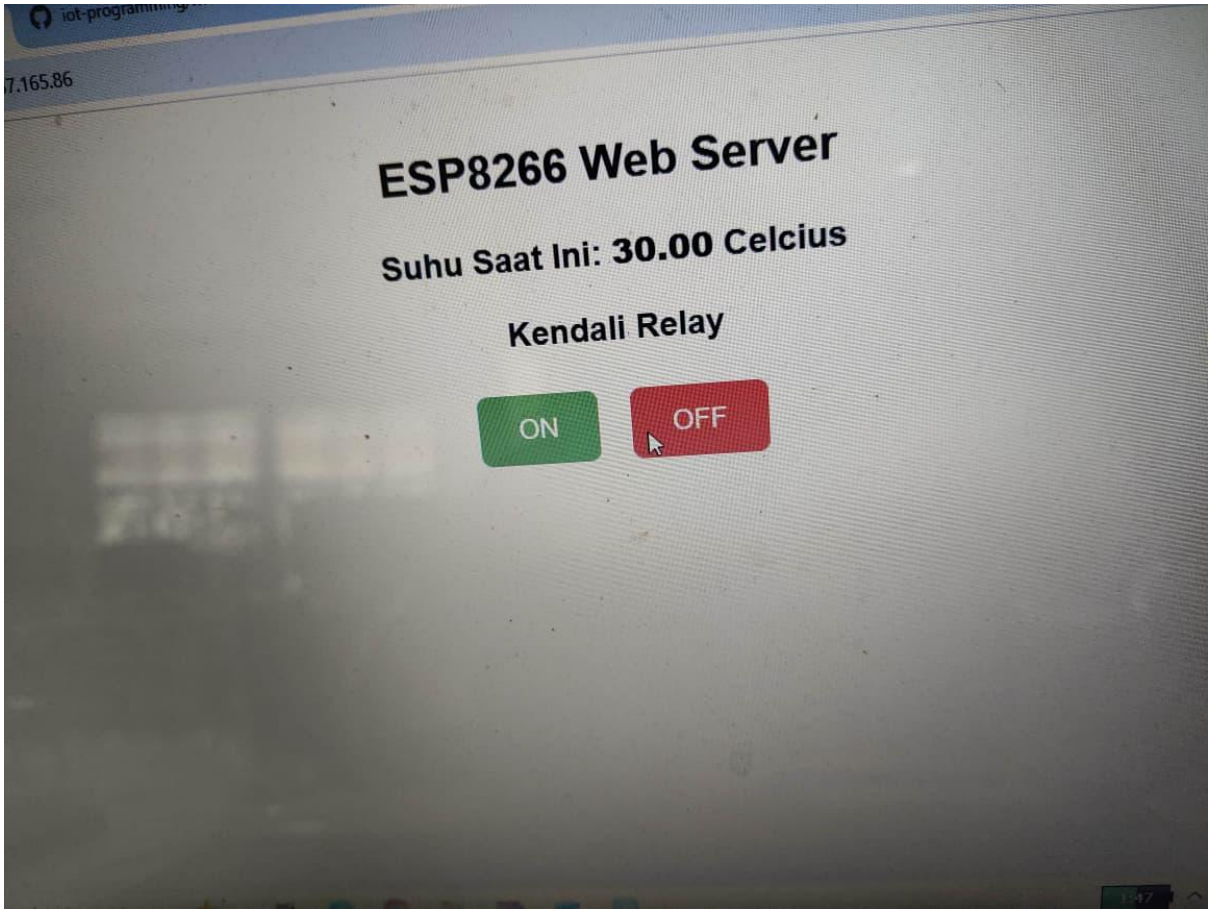
    Serial.print("HTTP Response code: ");
    Serial.println(httpResponseCode);
```

```
    http.end();
    delay(10000);
  }
  delay(2000);
}
```

3. Upload kode tersebut, dan buka Serial Monitor. Saat LDR Anda di *Client* ditutup tangan hingga gelap, perhatikan LED di NodeMCU *Server* milik teman Anda pasti akan menyala!


sketch_sep21b | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
sketch_sep21b.ino
void setup() {
Serial.begin(115200);
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
Serial.println("\nClient Terhubung ke Wi-Fi!");
}
void loop() {
int ldrValue = analogRead(ldrPin);
if ((WiFi.status() == WL_CONNECTED) && (ldrValue > 300)) {
WiFiClient client;
HTTPClient http;
http.begin(client, serverName);
int httpResponseCode = http.GET();
Serial.print("HTTP Response code: ");
Serial.println(httpResponseCode);
http.end();
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
New Line
115200 baud
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
Ln 22, Col 53 NodeMCU 1.0 (ESP-12E Module) on COM3
32°C Cerah
Search
16:34 21/09/2026


Praktikum_3_4 | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
Praktikum_3_4.ino
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <DHT.h>
const char* ssid = "KING_Emyu";
const char* password = "baktiari";
ESP8266WebServer server(80);
const byte dhtPin = 2;
const byte relayPin = 12;
DHT dht(dhtPin, DHT22);
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
New Line
115200 baud
WiFi Terhubung!
IP Address Server Anda:
10.14.211.86
Web Server dimulai!
indexing: 11/86
Ln 14, Col 1 NodeMCU 1.0 (ESP-12E Module) on COM3
Type here to search
14:55 22/09/2026



Latihan

Latihan 1 (Pemahaman Jaringan): Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*? saya menggunakan mode Station (STA) karena NodeMCU perlu terhubung ke Wi-Fi yang sudah tersedia, misalnya Wi-Fi HP atau router.

Pada praktikum ini, NodeMCU Server dan NodeMCU Client harus berada di jaringan Wi-Fi yang sama supaya Client bisa mengirim permintaan ke Server. Kalau menggunakan mode Access Point (AP), NodeMCU justru membuat jaringan Wi-Fi sendiri. Jadi, mode STA lebih sesuai dengan praktikum yang dilakukan.

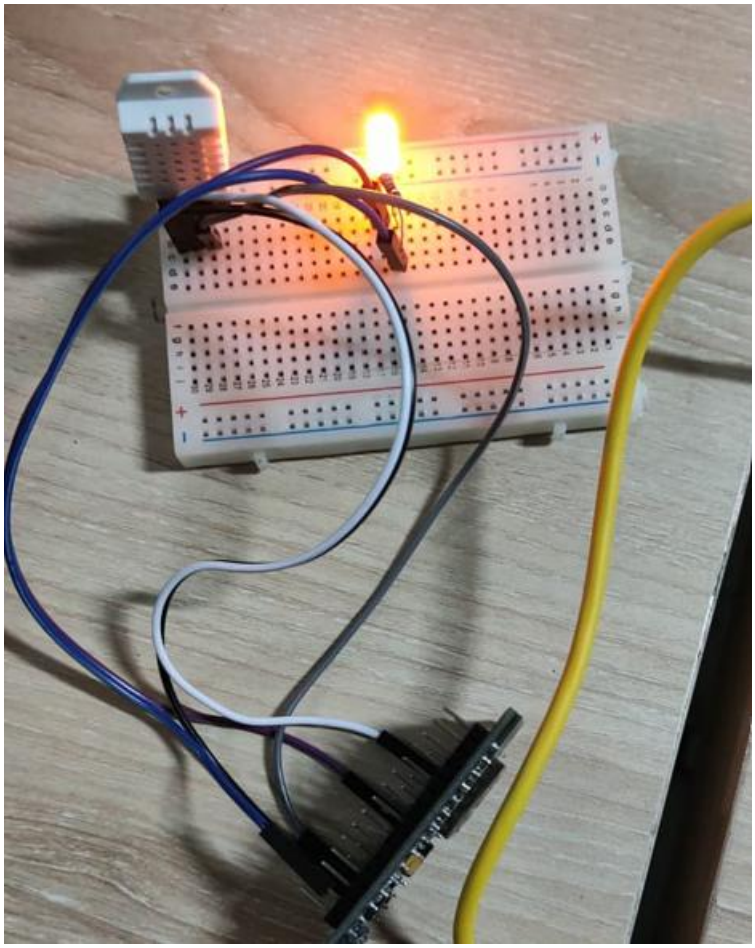
Latihan 2 (Modifikasi Antarmuka & Telemetri): Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:

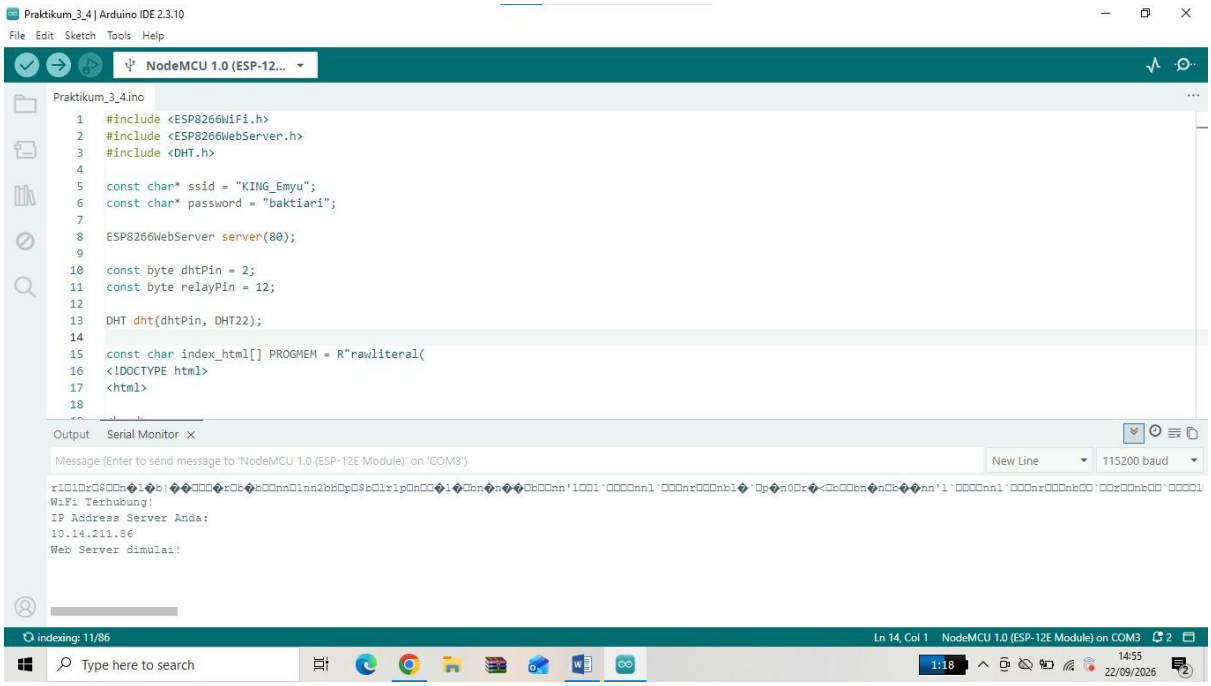
Menampilkan Kelembapan: Tambahkan pembacaan kelembapan menggunakan fungsi dht.readHumidity(). Sisipkan placeholder %HUMIDITY% pada kerangka HTML di dalam tag (contoh: Kelembapan: %HUMIDITY% %), lalu perbarui fungsi handleRoot() menggunakan html.replace("%HUMIDITY%", ...) agar nilai kelembapan muncul di halaman *web*.

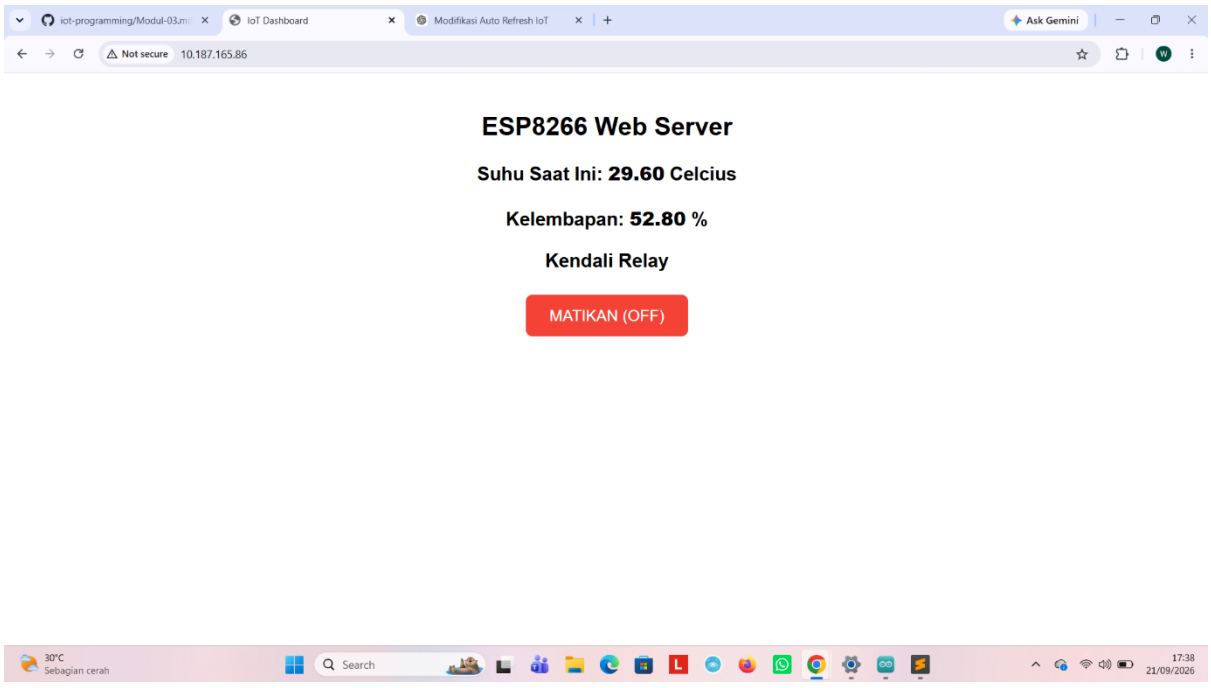
Tombol Dinamis Tunggal (*Toggle Button*): Ubah antarmuka agar hanya menggunakan 1 buah tombol tunggal yang statusnya adaptif mengikuti kondisi LED saat ini:

Jika LED sedang OFF, tombol menampilkan tulisan "NYALAKAN (ON)" (misal: warna hijau/biru) dan mengarahkan aksi ke /relay/on.

Jika LED sedang ON, tombol otomatis berubah menjadi tulisan "MATIKAN (OFF)" (warna merah) dan mengarahkan aksi ke /relay/off.
(Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti %RELAY_BUTTON%, lalu di fungsi handleRoot(), periksa status pin relay/LED menggunakan digitalRead(relayPin) sebelum mengganti placeholder tersebut dengan elemen tautan tombol yang sesuai).
Tuliskan potongan baris kode HTML dan fungsi handleRoot() yang Anda ubah!







Maksudnya:

- Kalau LED mati, tombol akan menjadi "NYALAKAN (ON)".
- Kalau LED menyala, tombol akan menjadi "MATIKAN (OFF)".

Jadi pengguna cukup menggunakan satu tombol saja untuk mengontrol LED.

Latihan 3 (Analisis Kode Client): Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata

NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).

jika NodeMCU Server offline, Serial Monitor Client akan menampilkan HTTP Response code dengan nilai negatif, contohnya -1. Hal ini terjadi karena Client tidak mendapatkan respons dari Server akibat Server kehilangan daya.



Jika NodeMCU Server teman saya tiba-tiba mati atau kehilangan daya, maka Client tidak bisa terhubung ke Server.

Latihan 4 (Pengembangan Algoritma): Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) HANYA JIKA sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

Baris algoritmanya adalah:

```
float temperature = dht.readTemperature();

if ((WiFi.status() == WL_CONNECTED) && (temperature > 35)) {
   // Kirim GET Request
}
```

Artinya: Client hanya mengirim GET Request jika suhu yang dibaca sensor DHT lebih dari 35°C dan Wi-Fi sedang terhubung. Kalau suhu ≤ 35°C, Client tidak mengirim request.

Tugas

Instruksi:

Lakukan modifikasi kode hanya pada sisi NodeMCU Server (Latihan 2).

Modifikasi variabel Raw Literal index_html. Sisipkan perintah *Meta Refresh HTML* (sebuah Tag standar web kuno namun ampuh) persis di dalam cakupan *tag* <head> untuk memaksa *browser* memuat ulang halaman secara sekuensial setiap 5 detik.

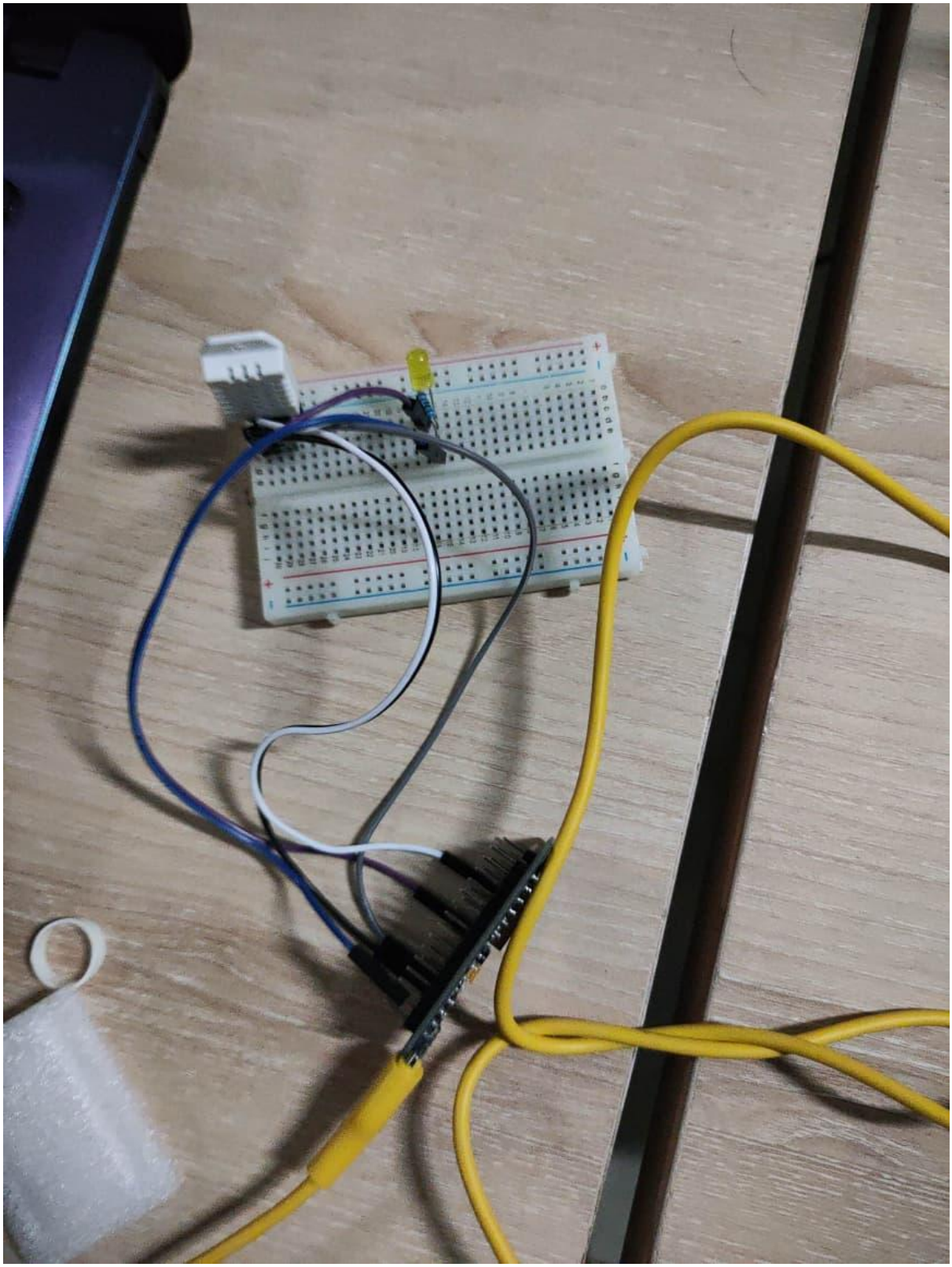
Hint (Petunjuk): Sintaks standar yang harus disisipkan adalah <meta http-equiv="refresh" content="5">.

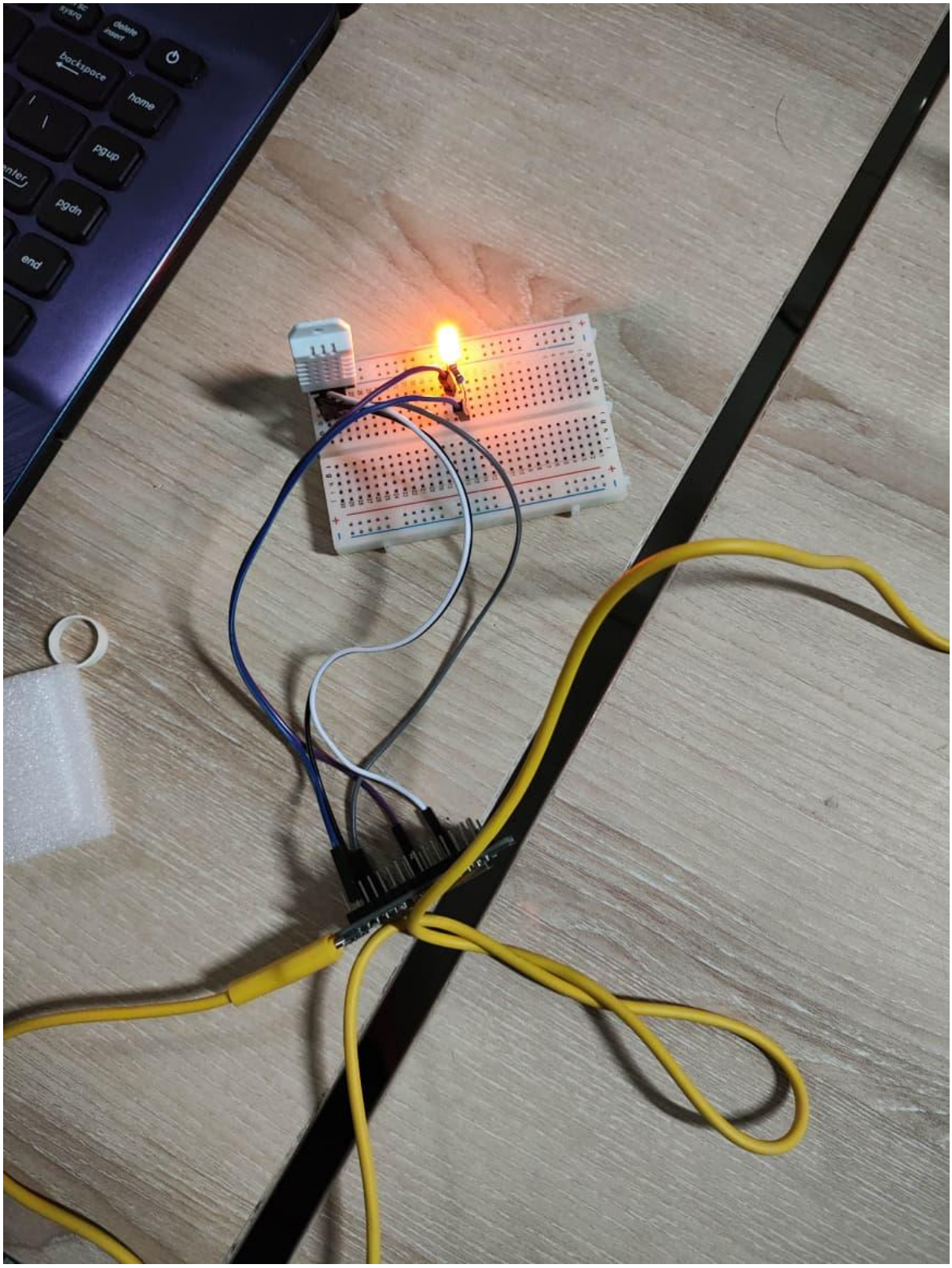
Setelah selesai, buka *browser*, dan amati layarnya. Biarkan selama 20 detik tanpa menyentuh *mouse/keyboard*.

Instruksi Analisis Arsitektur:

Setelah Auto-Refresh berjalan, cobalah tekan tombol *ON / OFF* (kendali LED) dari *browser*.

Apa kelemahan visual/interaksi (*flickering*) yang Anda alami saat menggunakan metode *Meta Refresh* HTTP konvensional seperti ini untuk sistem yang mengklaim diri sebagai *Real-Time Dashboard*? Tulis evaluasi logis Anda di Laporan Praktikum (Anda bisa membahas masalah *bandwidth*, layar berkedip putih, atau potensi bentrokan jika Anda mengeklik tepat di detik ke-5).




backspace
home
pgup
pgdn
end



ESP8266 Web Server

Suhu Saat Ini: 29.50 Celcius

Kelembapan: 52.90 %

Kendali Relay

MATIKAN (OFF)





ESP8266 Web Server

Suhu Saat Ini: 29.50 Celcius

Kelembapan: 52.70 %

Kendali Relay

NYALAKAN (ON)

